Investigating Divergence and Adaptations Between Heat Shock Proteins of Thermophilic Prokaryotes and Mesophilic Eukaryotes

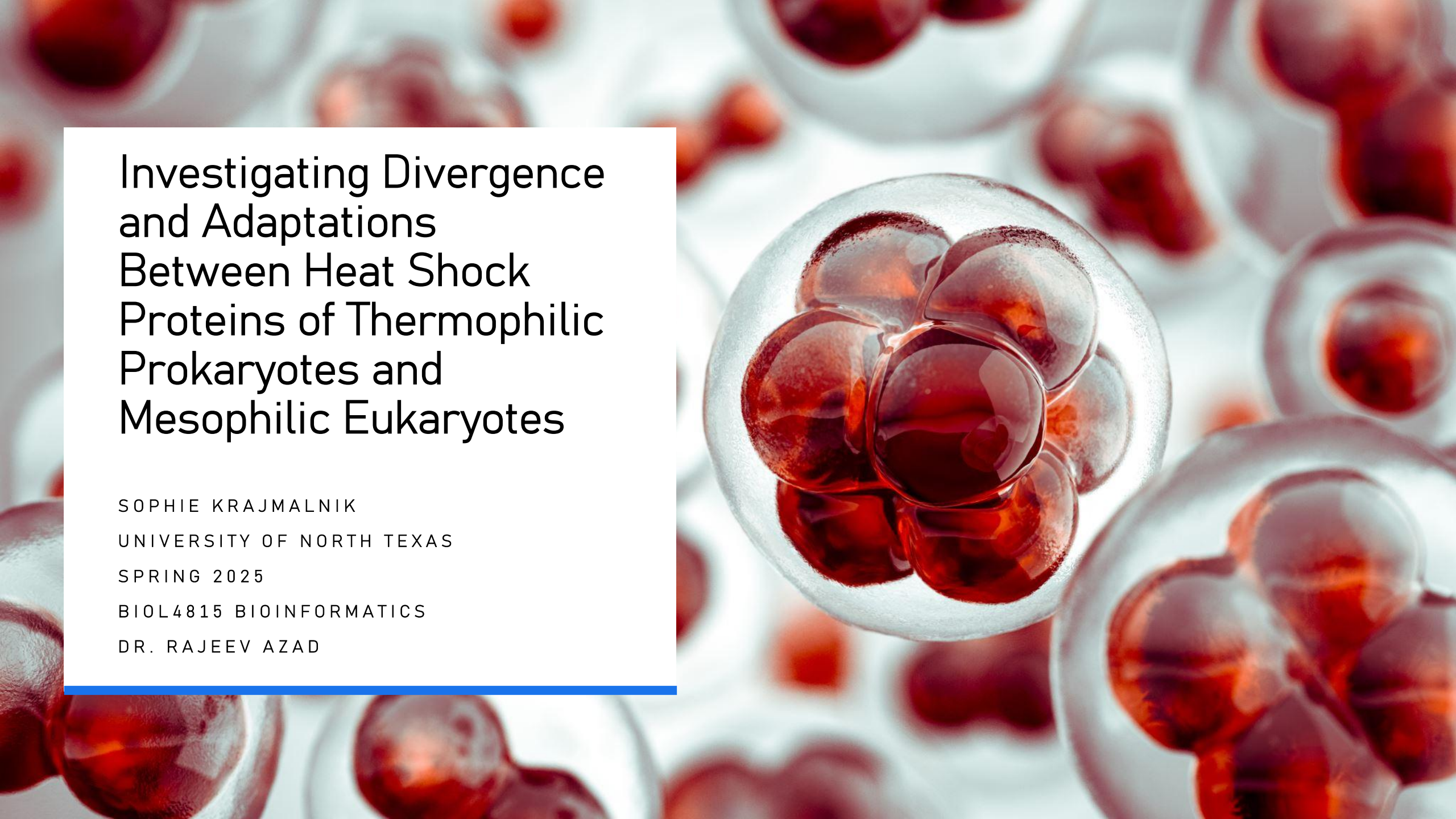
SOPHIE KRAJMALNIK

UNIVERSITY OF NORTH TEXAS

SPRING 2025

BIOL4815 BIOINFORMATICS

DR. RAJEEV AZAD





Extremophiles & HSPs

- Extremophiles:
  - Capable of surviving extreme environmental conditions ex. temperature, pressure, salinity, pH, radiation, and more
  - Structural, biochemical, and metabolic adaptations and strategies, due to selective pressures, provide these capabilities
  - Important for adaptation and biodiversity, biochemical catalysts, bioremediation, survivability in outer space, and more
  - Most extremophiles tend to be prokaryotes, due to their simplicity, ability for rapid evolution, and presence in early Earth conditions
- Heat Shock Proteins:
  - Help an organism respond to environmental stresses, such as heat, changes in oxygen content, DNA damage, and can activate apoptosis in stressful conditions (Hu et al., 2022)
  - In thermophilic prokaryotes, HSP abundance is related to thermostability (Trent et al., 1994); in mesophilic eukaryotes, HSP abundance leads to cancer, heart issues, and more systematic problems

Previous Research & Current Investigation

Previous research has examined HSPs in thermophiles and has performed analyses of GC content and codon usage bias at the genome-wide level rather than on specific genes like HSPs (Singer et al., 2003; Khan et al., 2018)

Phylogenetic analyses have been performed within a domain but rarely cross-domain (Gupta et al., 1994, 1997)

Project geared towards helping to bridge the gap between the similar HSP proteins between domains and environmental category through comprehensive, direct comparison

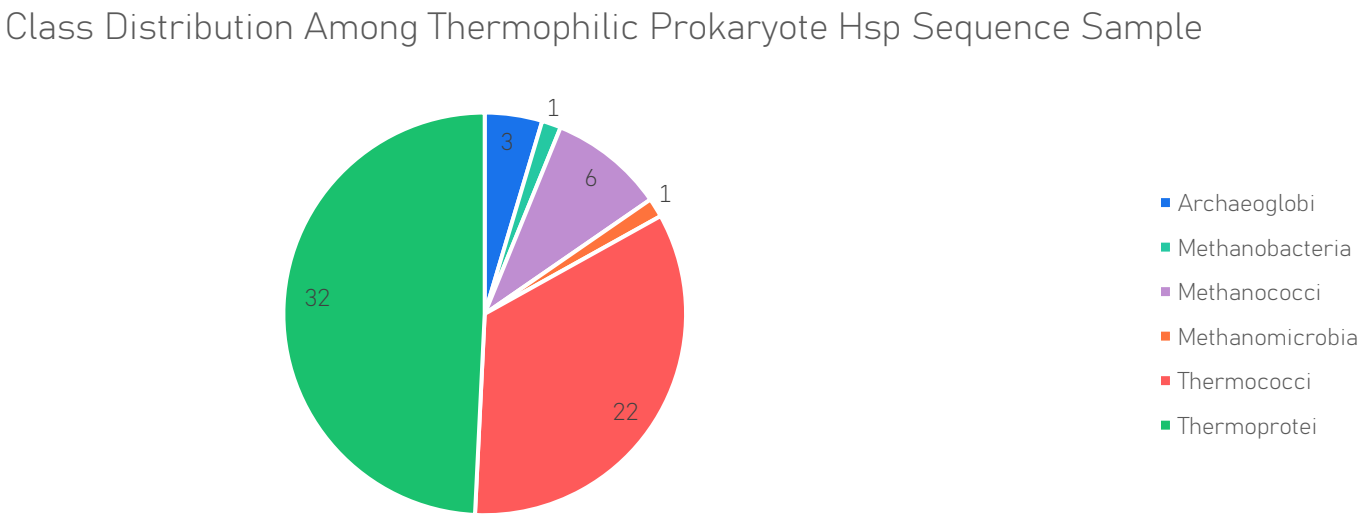
Goal: Investigate the possible adaptation and divergence of heat shock proteins between thermophilic prokaryotes and mesophilic eukaryotes

Provide insight into the mechanisms and evolutionary pressures that enable prokaryotic HSPs to withstand more heat, while eukaryotic HSPs contribute to negative pathological outcomes

Implement a variety of techniques learned in class integrated with RStudio

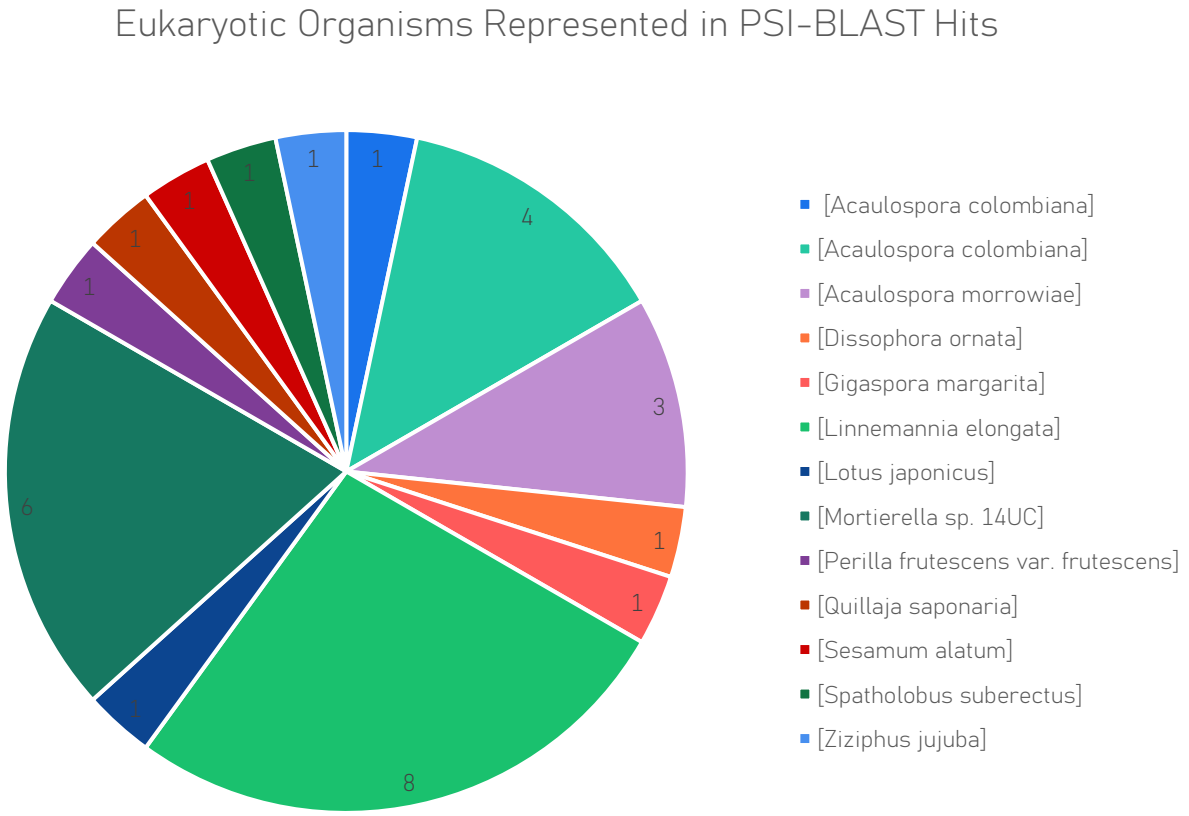
Organism and Sequence Acquisition

1. Take a sample of organisms from ThermoBase, a comprehensive database of thermophiles and hyperthermophiles, keeping track of Phylum and Class
2. From the above sample, take a representative sample of each class in the initial sample for a total of 15 thermophilic prokaryotes to analyze
    1. This number was chosen simply for feasibility in the given time span and to allow for at least 1 sample for each class – future work can expand on these numbers
3. Run each sample through NCBI to guarantee information is available and find an HSP sequence to focus on
    1. Since HSP20 emerged as a frequent pattern, this specific HSP was selected over others



Organism and Sequence Acquisition

1. Once each HSP20 sequence was selected for the prokaryotic thermophiles, run each sequence through PSI-BLAST with 6 iterations (parameters: database nr, restricted to Eukaryota, word size 3, 100 target sequences, BLOSUM62)
2. Select the top 2 hits from PSI-BLAST for each prokaryote
    1. Attempted to eliminate any partial or hypothetical proteins where possible

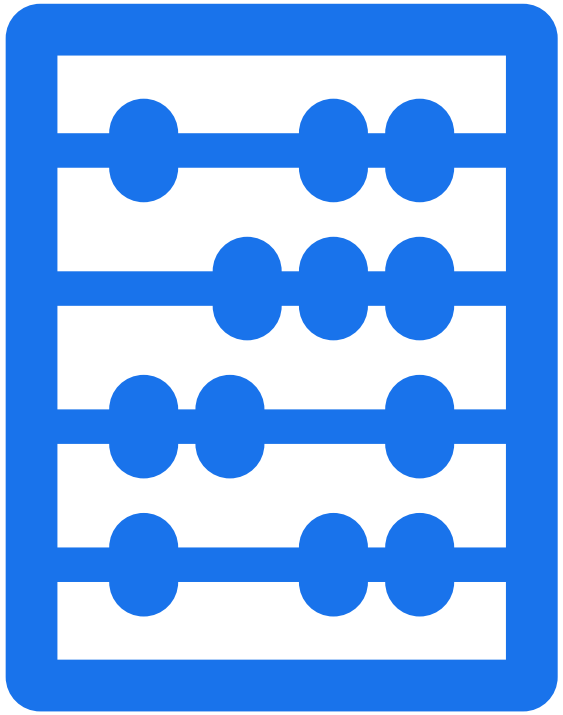


For 15 prokaryotic organisms, hits spanned only 12 different eukaryotic organisms – fungi (Glomeromycetes and Mortierellomycetes) and plants (Magnoliopsida)

PSI-BLAST Analysis

PSI-BLAST Results:
Avg Percent Identity = 24.5% (stdev = 2.9)
Avg E-Value = 3.33e-58 (stdev = 1.83e-57)
Avg Bit Score = 239 (stdev = 29.3)

- Expected low percent identity due to long evolutionary period of divergence between eukaryotes and prokaryotes
- Average percent identity drops into the twilight zone - expected due to distance, but attempting to pull in distant homologs (likely orthologs) within the protein family; could still share functionality
  - QUESTION: What is the rate of evolution? Could so many mutations have occurred that they have cycled back around to the same amino acids? Does this constant percent identity represent the constant rate of evolution within a protein family (molecular clock hypothesis)?
- Bit score and extremely low E-values suggest high quality alignments – conserved structure or domains



Multiple Sequence Alignment, Distance Matrix, and Principal Component Analysis

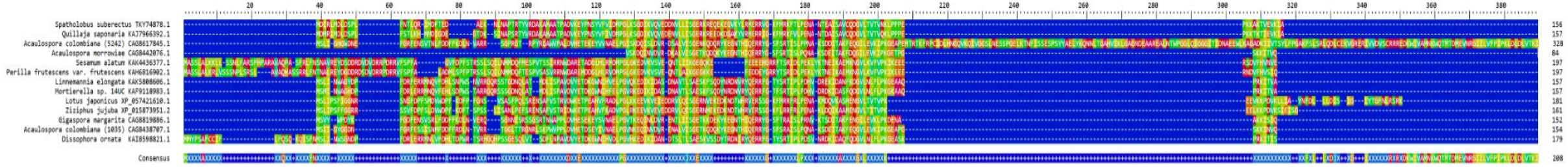
AMINO ACID LEVEL ANALYSIS

Multiple Sequence Alignment

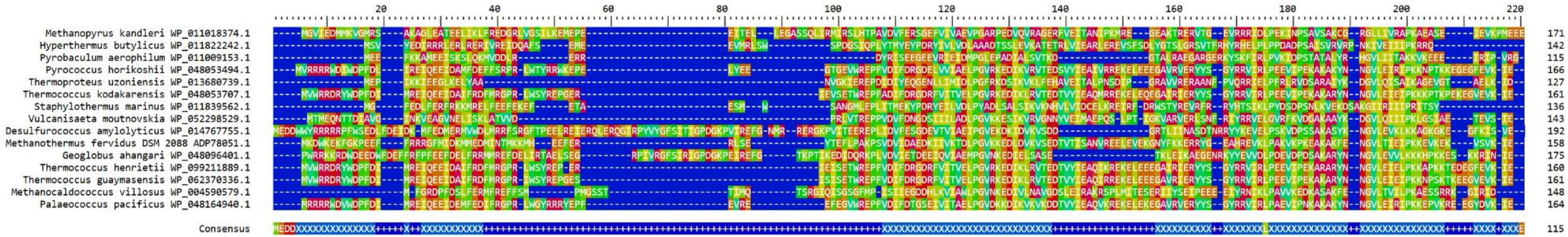
R Packages: Biostrings, DECIPHER

- Load amino acid sequences in FASTA format into AAStringSet object
- Call AlignSeqs function on the object of FASTA sequences – specifically designed to perform multiple sequence alignment → OUTPUT: sequences aligned and color-coded according to amino acid qualities; consensus sequence at bottom
  - Consensus sequence attempts to derive a "typical" sequence from the alignment, where X = unknown and + = conserved position
  - AlignSeqs by default calls the MUSCLE alignment method (iterative refinement)
  - Sequence alignment method can also be called on a DNAStringSet object (sequences of DNA), but protein selected as can be more informative
- Cross-Domain Multiple Sequence Alignment revealed remnants of shared ancestry
  - Positions with lots of variability in regions with a bulk alignment of amino acids
  - Conserved positions tend to correspond to gaps put in the alignment (low identity)– not as representative of protein function
- Eukaryotic Alignment revealed…
  - Clustering of alignment regions around positions 90 – 210 and 305-315
  - Higher in-domain similarity
- Prokaryotic Alignment revealed…
  - Clustering of alignment regions around positions 30-50 and 100-200 with much tighter construction
  - Much higher in-domain similarity
- All alignments showed high degrees of variability and most "positives" corresponding to gaps
- Variety among both C-terminus and N-terminus (extensions) to help with specialization

Eukaryotic Multiple Sequence Alignment



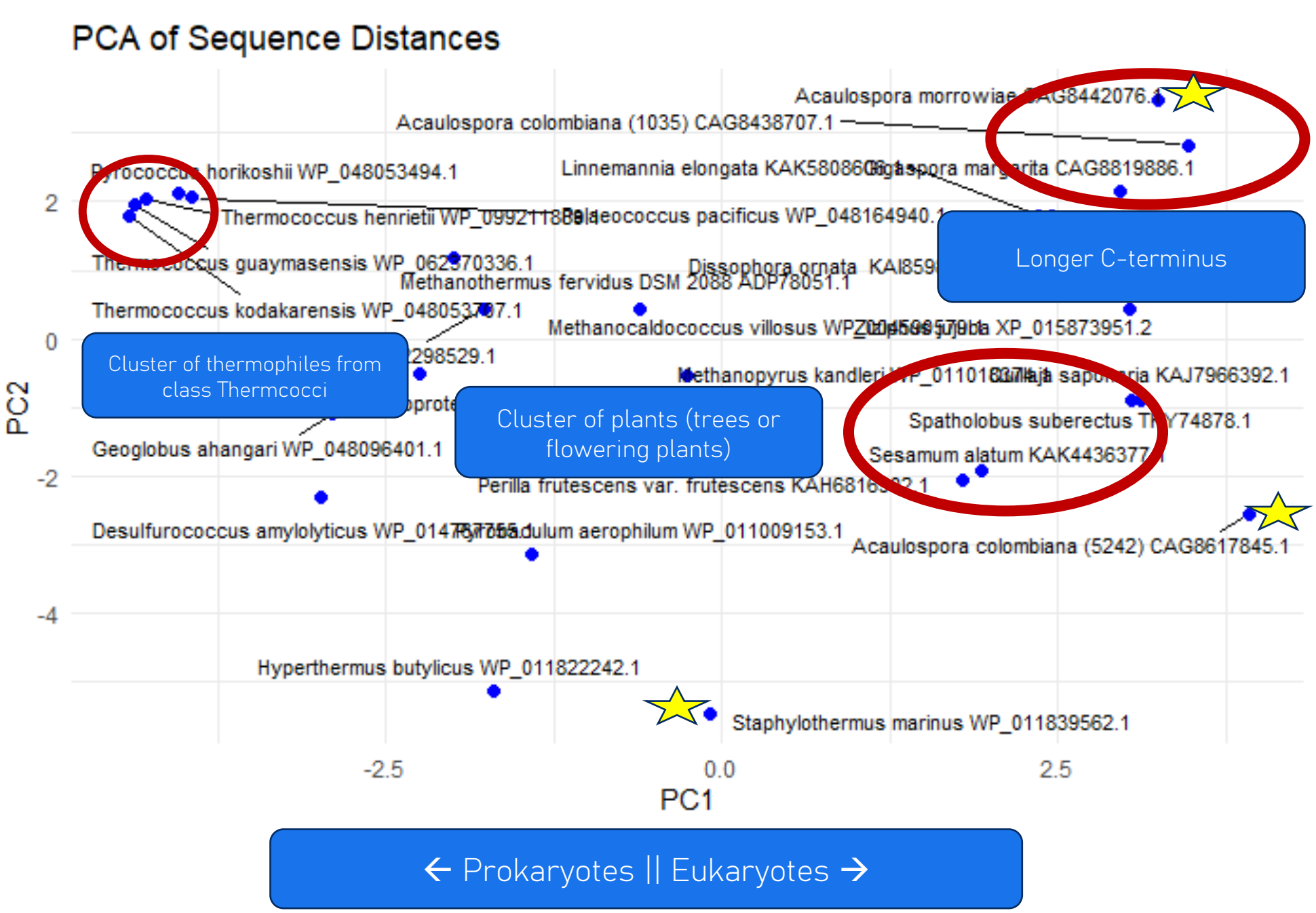
Prokaryotic Multiple Sequence Alignment



Distance Matrix & Principal Component Analysis

R Packages: DECIPHER, ggplot2, ggrepel

- Distance Matrix: pairwise distance ("difference") between two sequences in the multiple sequence alignment
  - Call the DistanceMatrix function with the Poisson correction applied – useful for amino acid alignments; accounts for multiple substitutions per site
  - OUTPUT: Distance values varying between [0, 3) – up to 3 amino acid substitutions per site (actual max = 2.88), at which point highly diverged
- Principal Component Analysis (PCA): dimensionality reduction and feature selection technique – identifies principal components that maximize variance (what are the main patterns of variation?)
  - Call prcomp from base R on the distance matrix generated previously, and standardize the distances (mean = 0, variance = 1)
  - X-axis is principal component 1 (largest source), with Y-axis as principal component 2 (second largest source)
  - Shows tighter clustering among the prokaryotes (less distance, more conservation) than among the eukaryotes, but the groups are separated



PCA Analysis:

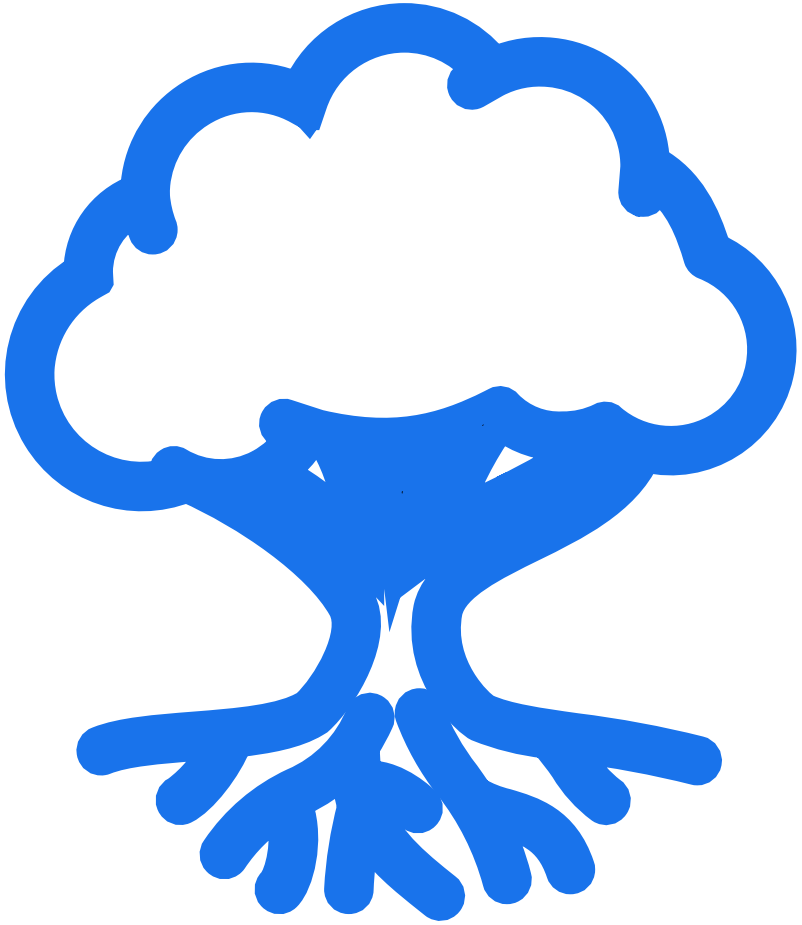
PCA1 contributes to 30.97% of the variance; PCA2 contributes to 19.15% of the variance – overall half of variance (50.12%)

Since working off a distance matrix, specific features aren't represented (genes, amino acids, etc.)

Possible interpretations of PC1/2: domain, environmental category, overall sequence length

Can provide insight by seeing where clusters occur and which sequences are at extremes of each principal component

Can analyze extremes for further insight



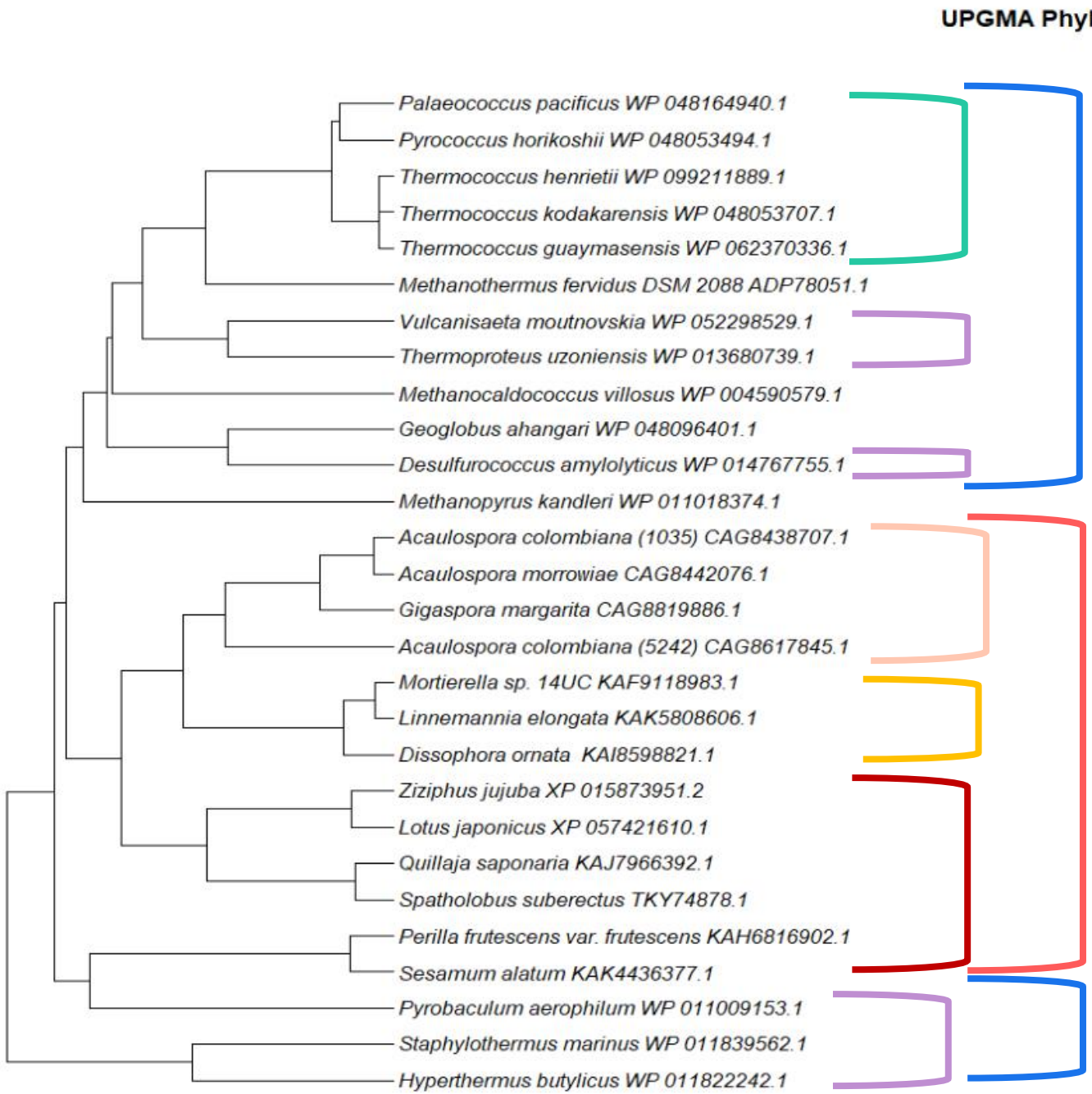
Phylogenetic Tree Generation

AMINO ACID LEVEL ANALYSIS

Phylogenetic Tree Generation & Bootstrapping

R Packages: ape, phangorn

- Generate and plot phylogenetic tree based on the created distance matrix using UPGMA method
  - While could introduce errors since organisms can evolve at different rates, effective method for creating a tree based on distance and can help explore the possibility of application of molecular clock hypothesis
  - Future work can attempt model or character-based techniques
- Generates reasonable tree, following known taxonomy
  - Prokaryotes separated at bottom could be due to faster evolutionary rate
- Run the bootstrap method 100 times on the initial phylogenetic tree with the distance matrix by calling boot.phylo. Using the bootstrap values, call consensus to build the consensus tree, where branches must be included in at least 39% of trees. Plot the tree with the confidence values.
  - Generally supports the initial tree, but most prokaryotic clades were removed due to low confidence values – noise introduced (gaps, tails) or issue with the method used
  - 39% confidence is a low threshold for an actual analysis, but relatively high for visualization purposes here – 39% was the lowest to be able to perform a properly-structured tree
- Ran same process on tree with neighbor-joining method and threshold of 45%
  - Might introduce error due to questionable additivity of distances



Prokaryotic

Class: Thermococci

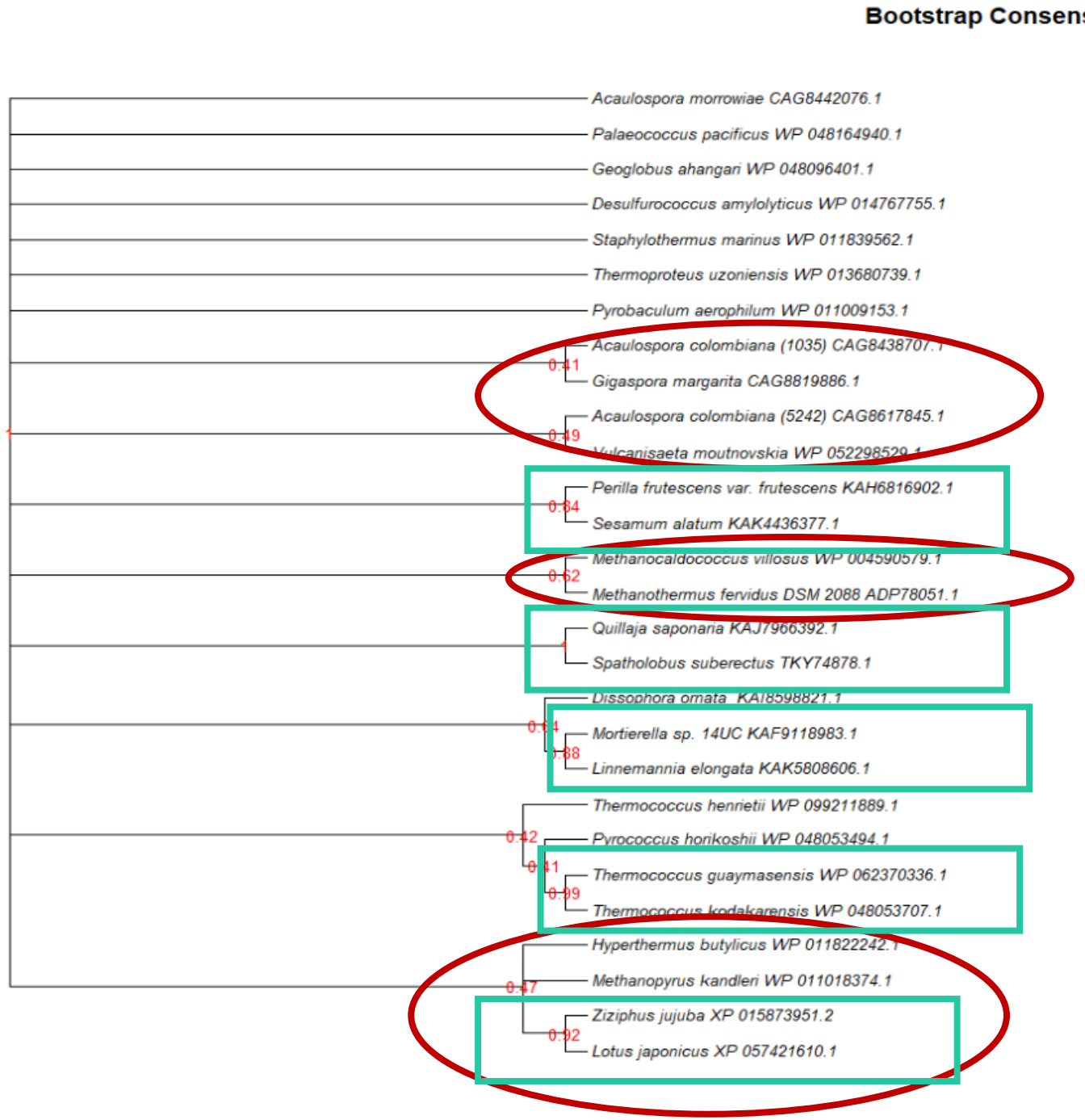
Class: Thermoprotei

Eukaryotic

Class: Glomeromycetes

Class: Mortierellomycetes

Class: Magnoliopsida



RESULTS:

Generally follows the branching logic and separation by domain indicated in initial tree

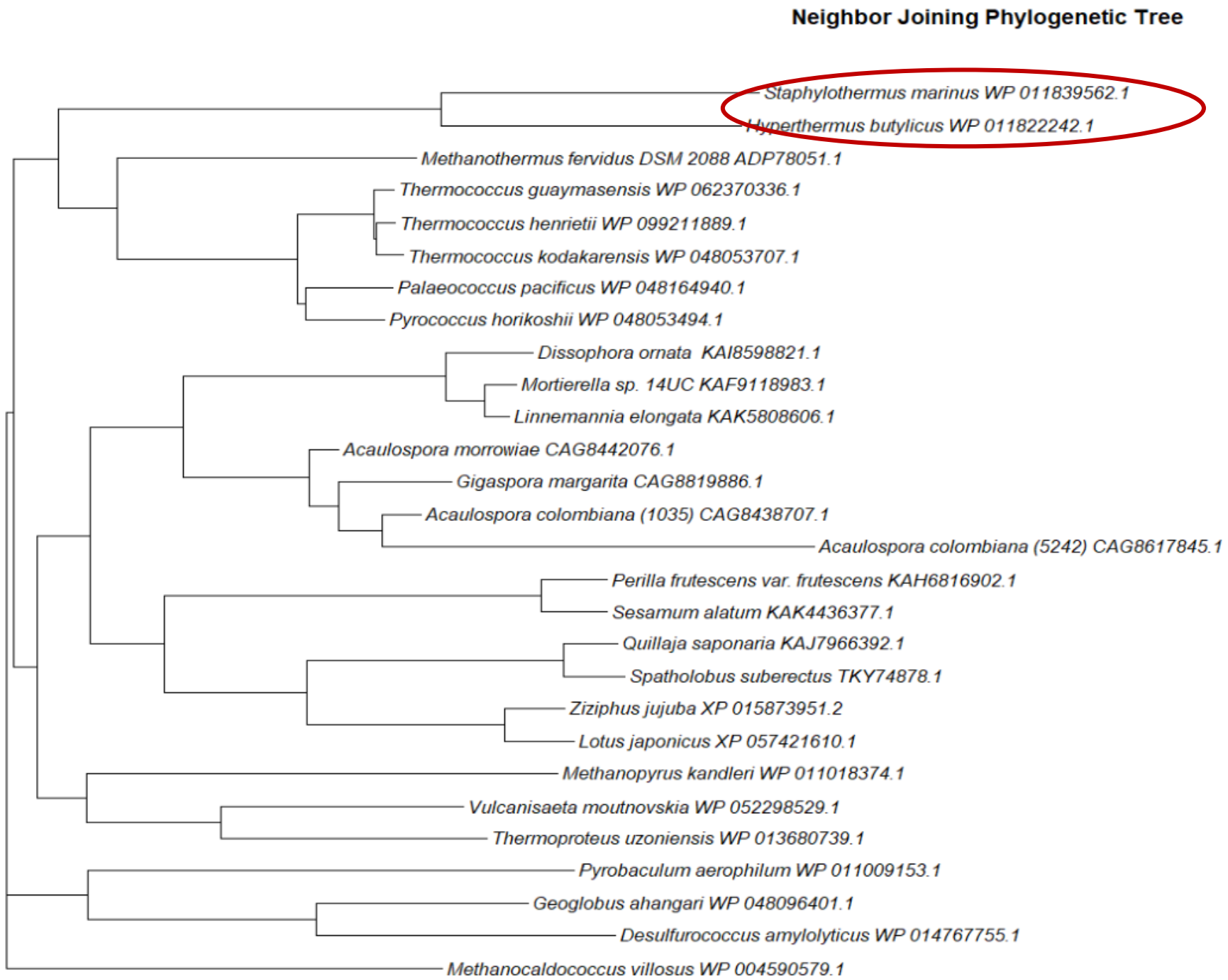
Interesting clade seen between two previously unconnected species from classes Methanococci and Methanobacteria – logical

Strong support for eukaryotic clades

Weaker support for prokaryotic clades caused them to disappear in consensus – interesting because higher identity

Interesting clade of Thermoprotei, Methanomicrobia, Magnoliopsida

Separated the two Acaulospora colombiana sequences

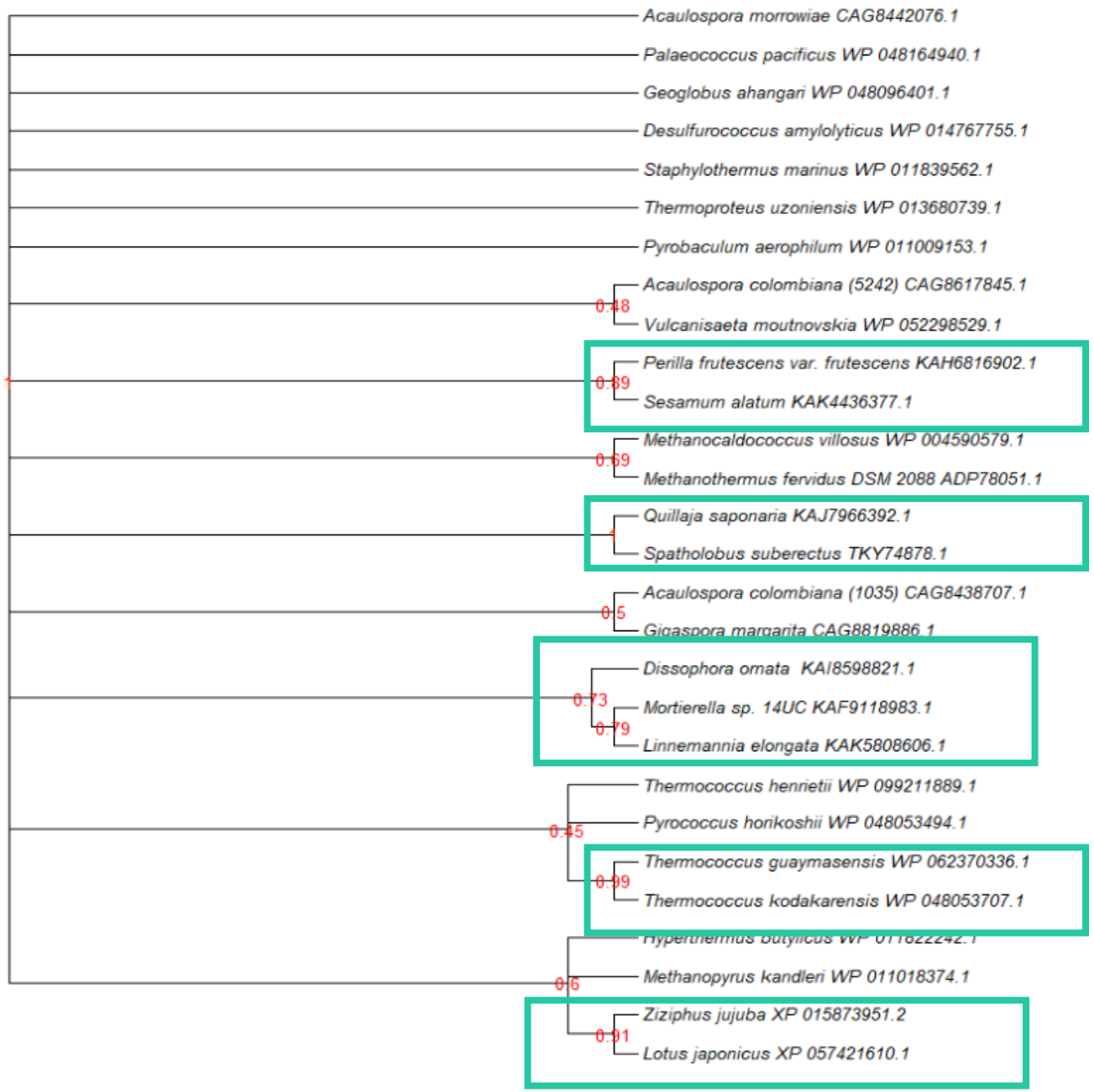


RESULTS:
Moved likely quickly evolving prokaryotes to the top of the tree

Overall structure is similar but with a difference in the order of branching

Plant clade successfully identified

Bootstrap Consensus Tree- Neighbor-Joining



RESULTS:

Recovered same clades as UPGMA, but in slightly different order

Overall higher confidence percentage

Same recovery of interesting clade pairs (cross-domain, separating Acaulospora colombiana, combining Methanobacteria and Methanococci)



GC Content Analysis

DNA LEVEL ANALYSIS

GC Content Analysis

R Packages: Biostrings

- Load DNA sequences into DNAStringSet object. Call letterFrequency, focusing on letters G and C, on the DNA sequences, and store as proportion. Sum the G proportion and the C proportion to get overall GC content per sequence.
  - RESULTS:

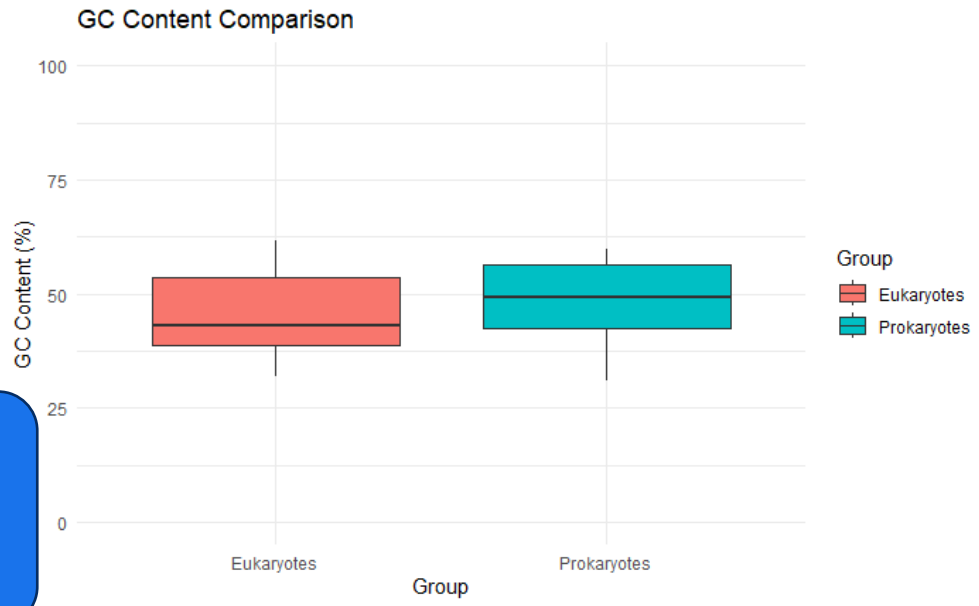
```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.3082  0.4061  0.4495  0.4667  0.5493  0.6166
```

  Only one sequence is GC-rich *(Perilla frutescens var. frutescens)*
- Process repeated, but with objects where sequences are separated into eukaryotes and prokaryotes → Plotted as boxplot to show difference
  - Higher median GC content in thermophilic prokaryotes
  - Positive skew for eukaryotes, while no skew for prokaryotes
    - Check the codon usage; tails; specific class

```
        Welch Two Sample t-test

data:  gc_prok and gc_euk
t = 0.73931, df = 25.928, p-value = 0.4664
alternative hypothesis: true diff
95 percent confidence interval:
 -4.654066  9.881260
sample estimates:
mean of x mean of y
 47.88471  45.27111
```

Using a two sample t-test, there was no statistically significant different in GC content between the thermophilic prokaryotes and the mesophilic eukaryotes. The mean GC content was 47.88% in prokaryotes, and 45.27% in eukaryotes, with a 95% confidence interval ranging from -4.65% to +9.88%.





Codon Usage Bias

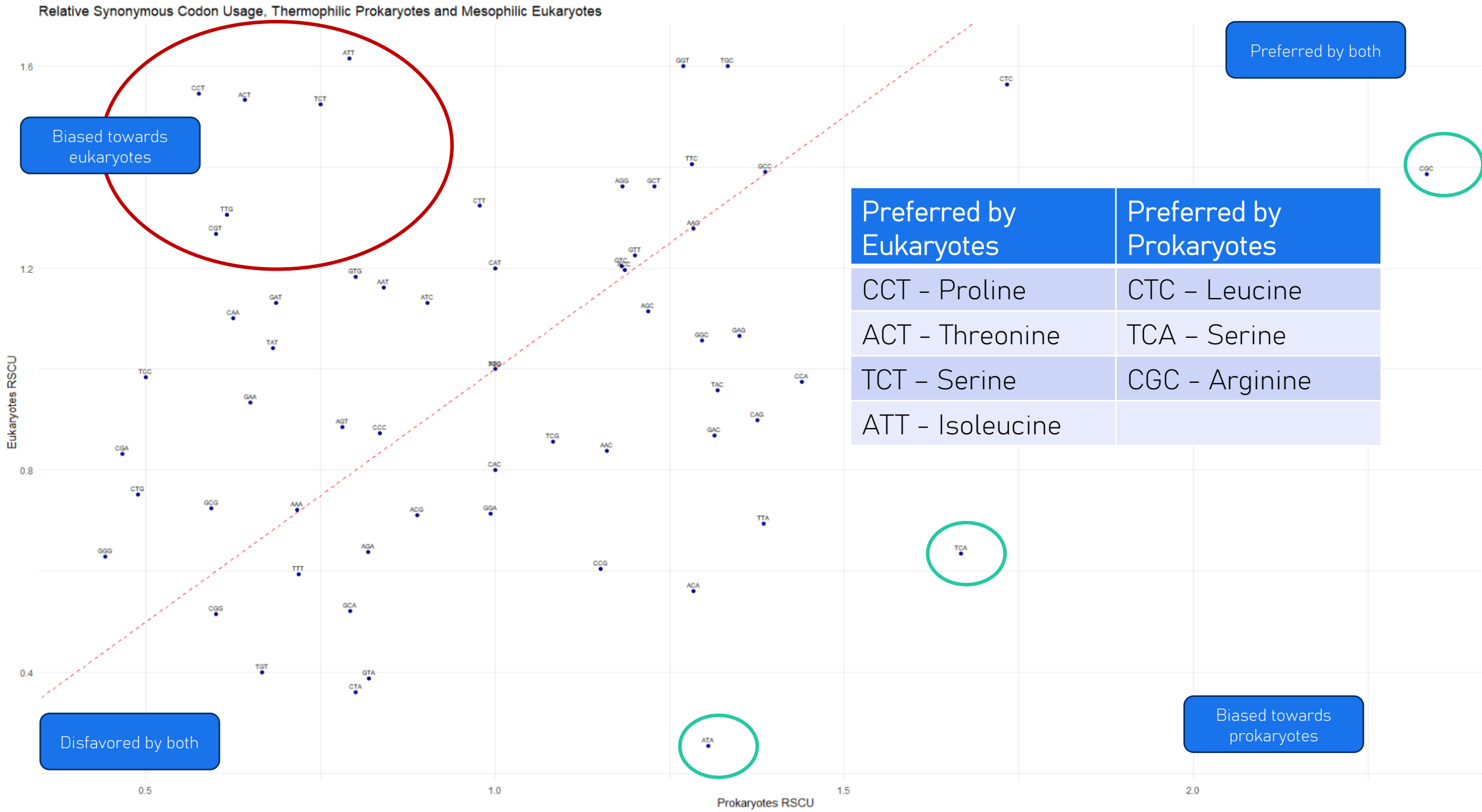
CDS LEVEL ANALYSIS

Relative Synonymous Codon Usage

R Packages: Biostrings, cubar, ggplot2

- Relative Synonymous Codon Usage (RSCU): observed frequency of codon / expected frequency of codon (assume equal usage for synonymous codons) – used to measure codon usage bias
- Read the CDS into DNAStringSet objects, one for each domain. Clean the sequences and prepare for use with cubar. Call count_codons to create data frames with the codon frequencies, then call est_rscu to estimate RSCU for each domain group. Combine into a data frame, and plot as a scatterplot.
  - Points on the guideline are used equally in the prokaryotes and eukaryotes
    - Above the line = biased towards eukaryotes (CCT, ACT, TCT, ATT)
    - Below the line = biased towards prokaryotes (CTC, TCA, CGC)
  - Further work: Map back onto the genes to see domains

Preferred by Eukaryotes	Preferred by Prokaryotes
CCT - Proline	CTC – Leucine
ACT - Threonine	TCA – Serine
TCT – Serine	CGC - Arginine
ATT - Isoleucine	



Preferred by Eukaryotes	Preferred by Prokaryotes
CCT – Proline	CTC – Leucine
ACT – Threonine	TCA – Serine
TCT – Serine	CGC – Arginine
ATT – Isoleucine	



Conclusions

Conclusions

- MSA revealed domains of amino acids with high variability and some reasonably “alignable” domains
  - N- and C- terminus possibilities with high impact on output in MSA and PCA
- PCA revealed domain is highest source of variation, as expected – analyze extremes on all axes
  - More variety within group of eukaryotes than in group of prokaryotes (tighter clustering)
- Phylogenetic tree pulled out interesting series of combinations and separations between domain, class, and species that are worth studying
  - Importance of algorithm selection, specifically as related to evolutionary rate
- GC content showed no statistically significant difference cross-domain – not related to function
- RSCU revealed different codons biased towards each domain group - analyze

Future Work

Improvements:

- Statistically significant sample sizes
- Run reciprocal PSI-BLAST to ensure meaningful sequence selection

Different Analyses:

- Entropy analysis
- Motif analysis
- Protein-folding
- Apply machine learning
- Techniques to decrease impact of domain
- Profile HMM at termini, domain, or core level

Focus:

- Specific classes
- Extend to Animals
- Analyze the termini
- Remove termini and focus on core

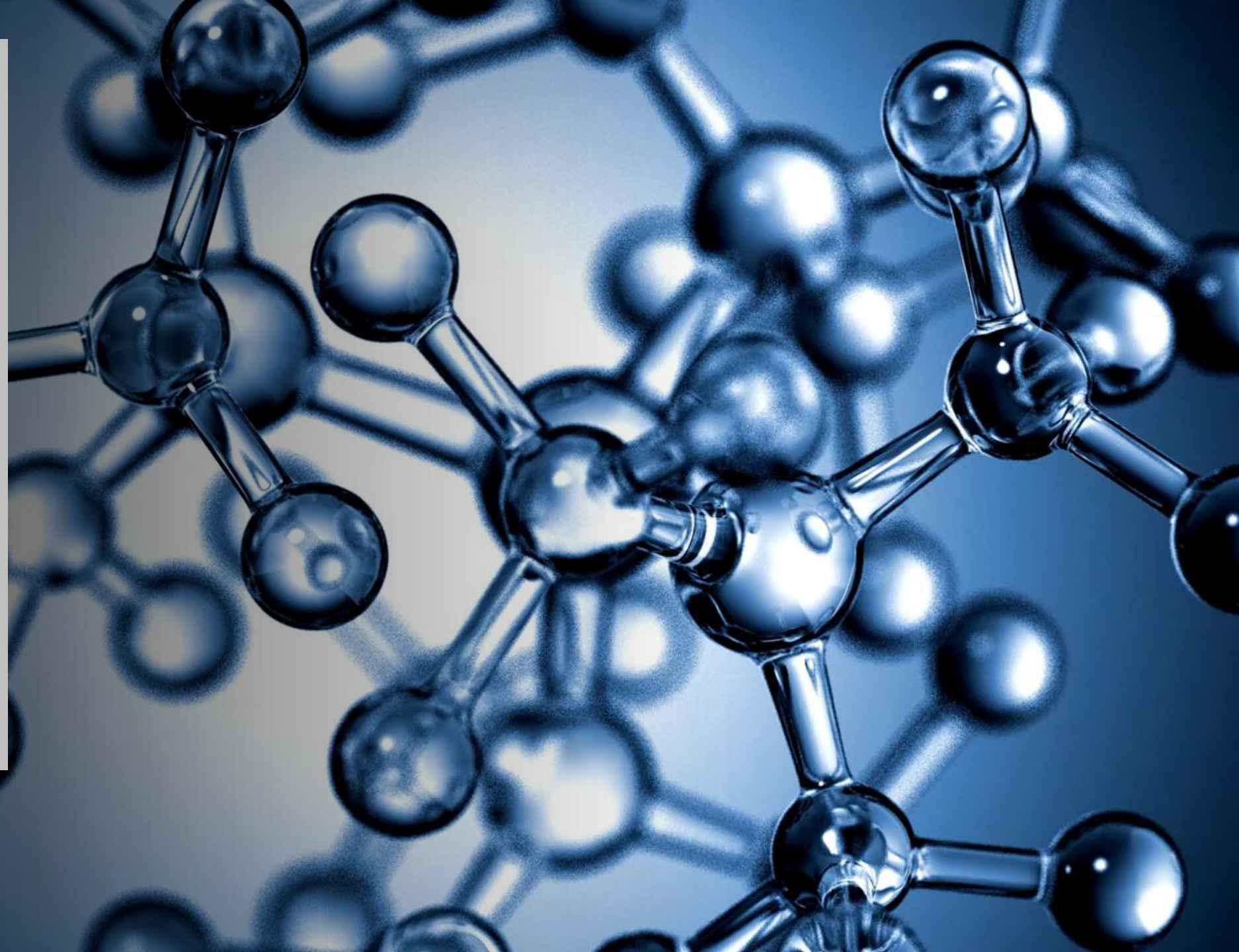
While nothing completely revolutionary was discovered, it was interesting to have a practical application of so many different bioinformatics techniques in conjunction & gain further appreciation

Pull out ideas for future analyses

Combine the techniques with validation methods and visualizations

Automation of so many of these functions through variety of R packages

- PROS: easy to perform, quick, integrated environment, variety of open-source packages
- CONS: harder to determine sources of error, learning curve



Bibliography

DiGiaocomo, J., McKay, C., & Davila, A.
(n.d.). ThermoBase: A Centralized Database of Thermophiles and Hyperthermophiles. Thermobase. https://togodb.org/db/thermobase

Gupta RS, Bustard K, Falah M, Singh D.
1997.Sequencing of heat shock protein 70 (DnaK) homologs from Deinococcus proteolyticus and Thermomicrobium roseum and their integration in a protein-based phylogeny of prokaryotes. J Bacteriol179:.https://doi.org/10.1128/jb.179.2.345-357.1997

Gupta, R. S., & Singh, B.
(1994). Phylogenetic analysis of 70 kD heat shock protein sequences suggests a chimeric origin for the eukaryotic cell nucleus. Current biology : CB, 4(12), 1104–1114. https://doi.org/10.1016/s0960-9822(00)00249-9

H B Rappaport, Angela M Oliverio,
Lessons from Extremophiles: Functional Adaptations and Genomic Innovations across the Eukaryotic Tree of Life, Genome Biology and Evolution, Volume 16, Issue 8, August 2024, evae160, https://doi.org/10.1093/gbe/evae160

Hu, C., Yang, J., Qi, Z., Wu, H., Wang, B., Zou, F., Mei, H., Liu, J., Wang, W., & Liu, Q.
(2022). Heat shock proteins: Biological functions, pathological roles, and therapeutic opportunities. MedComm, 3(3), e161. https://doi.org/10.1002/mco2.161

Hurst, L. D., & Merchant, A. R.
(2001). High guanine-cytosine content is not an adaptation to high temperature: a comparative analysis amongst prokaryotes. Proceedings. Biological sciences, 268(1466), 493–497. https://doi.org/10.1098/rspb.2000.1397

Khan, M. F., & Patra, S.
(2018). Deciphering the rationale behind specific codon usage pattern in extremophiles. Scientific reports, 8(1), 15548. https://doi.org/10.1038/s41598-018-33476-x

Kumar, S., Tamura, K., & Nei, M.
(1993). Basic Sequence Statistics. Retrieved from MEGA Manual: https://www.megasoftware.net/mega1_manual/BasicStats.html

Pagès H, Aboyoun P, Gentleman R, DebRoy S (2024). Biostrings: Efficient manipulation of biological strings. R package version 2.74.1, https://bioconductor.org/packages/Biostrings.

Paradis E, Schliep K (2019). "ape 5.0: an environment for modern phylogenetics and evolutionary analyses in R." Bioinformatics, 35, 526-528. doi:10.1093/bioinformatics/bty633.

Parvathy, S. T., Udayasuriyan, V., & Bhadana, V. (2022). Codon usage bias. *Molecular biology reports*, *49*(1), 539–565. https://doi.org/10.1007/s11033-021-06749-4

Rampelotto P. H. (2013). Extremophiles and extreme environments. Life (Basel, Switzerland), 3(3), 482–485. https://doi.org/10.3390/life3030482

Schliep K.P. 2011. phangorn: phylogenetic analysis in R. Bioinformatics, 27(4) 592-593

Singer, G. A., & Hickey, D. A.
(2003). Thermophilic prokaryotes have characteristic patterns of codon usage, amino acid composition and nucleotide content. Gene, 317(1-2), 39–47. https://doi.org/10.1016/s0378-1119(03)00660-7

Somayaji, A., Dhanjal, C. R., Lingamsetty, R., Vinayagam, R., Selvaraj, R., Varadavenkatesan, T., & Govarthanan, M.
(2022). An insight into the mechanisms of homeostasis in extremophiles. Microbiological research, 263, 127115. https://doi.org/10.1016/j.micres.2022.127115

Trent, J. D., Gabrielsen, M., Jensen, B., Neuhard, J., & Olsen, J.
(1994). Acquired thermotolerance and heat shock proteins in thermophiles from the three phylogenetic domains. Journal of bacteriology, 176(19), 6148–6152. https://doi.org/10.1128/jb.176.19.6148-6152.1994

Wright ES
(2016). "Using DECIPHER v2.0 to Analyze Big Biological Sequence Data in R." The R Journal, 8(1), 352-359.

Zhang H, Liu M, Zi B
(2024). cubar: Codon Usage Bias Analysis. R package version 1.1.0.9000, https://github.com/mt1022/cubar

Appendix

CODE, IMAGES, RAW DATA, AND ADDITIONAL INFORMATION

PSI-BLAST Results

Organism	Accession Number	Class	Name	Hsp Ty	Hit Accession N	Hit Organism	% Identity	E-value	Bit Score
Methanopyrus kandleri	WP_011018374.1	Methanomicrobia	17.9 kDa class II heat shock protein	p23	TKY74878.1	[Spatholobus suberectus]	24	3.00E-78	239
			17.3 kDa class II heat shock protein-like	p23	KAJ7966392.1	[Quillaja saponaria]	21	3.00E-78	239
Hyperthermus butylicus	WP_011822242.1	Thermoprotei	5242_t:CDS:2		CAG8617845.1	[Acaulospora colombiana]	18	4.00E-75	235
			8747_t:CDS:2		CAG8442076.1	[Acaulospora morrowiae]	18	3.00E-74	227
Pyrobaculum aerophilum	WP_011009153.1	Thermoprotei	Heat shock protein, mitochondrial		KAK4436377.1	[Sesamum alatum]	23	2.00E-60	193
			HSP20-like chaperones superfamily protein		KAH6816902.1	[Perilla frutescens var. frutescens	25	1.00E-56	183
Pyrococcus horikoshii	WP_048053494.1	Thermococci	HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	26	3.00E-93	277
			Heatshock protein HSP 90-beta		KAF9118983.1	[Mortierella sp. 14UC]	25	3.00E-92	274
Thermoproteus uzoniensis	WP_013680739.1	Thermoprotei	Heatshock protein HSP 90-beta		KAF9118983.1	[Mortierella sp. 14UC]	26	1.00E-70	217
			HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	25	8.00E-70	215
Thermococcus kodakarensis	WP_048053707.1	Thermococci	HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	26	2.00E-94	279
			Heatshock protein HSP 90-beta		KAF9118983.1	[Mortierella sp. 14UC]	25	7.00E-93	275
Staphylothermus marinus	WP_011839562.1	Thermoprotei	5242_t:CDS:2		CAG8617845.1	[Acaulospora colombiana]	22	4.00E-73	230
			8747_t:CDS:2		CAG8442076.1	[Acaulospora morrowiae]	19	2.00E-67	210
Vulcanisaeta moutnovskia	WP_052298529.1	Thermoprotei	18.5 kDa class I heat shock protein-like		XP_057421610.1	[Lotus japonicus]	25	1.00E-65	207
			18.5 kDa class I heat shock protein		XP_015873951.2	[Ziziphus jujuba]	26	5.00E-65	204
Desulfurococcus amylolyticus	WP_014767755.1	Thermoprotei	44068_t:CDS:2		CAG8819886.1	[Gigaspora margarita]	25	1.00E-63	202
			1035_t:CDS:2		CAG8438707.1	[Acaulospora colombiana]	21	2.00E-63	202
Methanothermus fervidus DSM2088	ADP78051.1	Methanobacteria	8747_t:CDS:2		CAG8442076.1	[Acaulospora morrowiae]	26	5.00E-87	260
			1035_t:CDS:2		CAG8438707.1	[Acaulospora colombiana]	30	2.00E-85	256
Geoglobus ahangari	WP_048096401.1	Archaeoglobi	HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	25	2.00E-74	229
			HSP20-like chaperone		KAI8598821.1	[Dissophora ornata]	26	9.00E-74	228
Thermococcus henrietii	WP_099211889.1	Thermococci	5242_t:CDS:2		CAG8617845.1	[Acaulospora colombiana]	21	2.00E-79	247
			HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	25	3.00E-80	243
Thermococcus guaymasensis	WP_062370336.1	Thermococci	HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	26	1.00E-93	277
			Heatshock protein HSP 90-beta		KAF9118983.1	[Mortierella sp. 14UC]	25	2.00E-92	274
Methanocaldococcus villosus	WP_004590579.1	Methanococci	Heatshock protein HSP 90-beta		KAF9118983.1	[Mortierella sp. 14UC]	29	1.00E-83	251
			HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	28	2.00E-83	251
Palaeococcus pacificus	WP_048164940.1	Thermococci	HSP20-like chaperone		KAK5808606.1	[Linnemannia elongata]	27	8.00E-94	278
			Heatshock protein HSP 90-beta		KAF9118983.1	[Mortierella sp. 14UC]	26	9.00E-94	278

Code for Multiple Sequence Alignment

```
## Multiple Sequence Alignment and Phylogenetic Tree
```

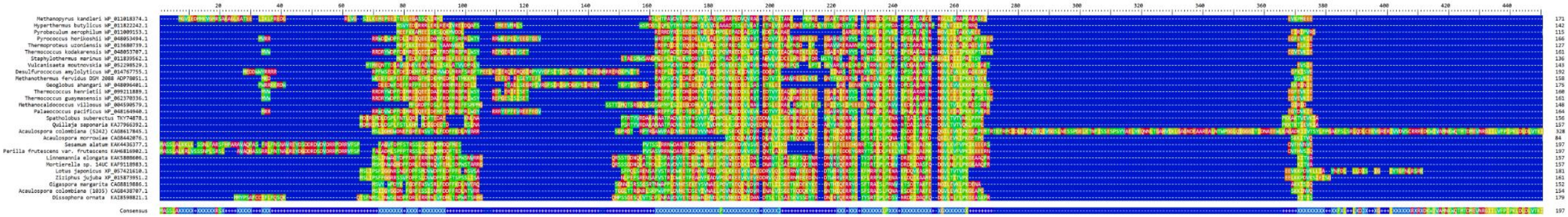
```{r}
sequences <- readAAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\projectfastaproteinseq.txt")
  class(sequences)
sequences ##Read in all sequences in FASTA format to AAStringSet object, and verify process

aligned <- AlignSeqs(sequences) ##Align sequences and store as AAStringSetObject
  class(aligned)
BrowseSeqs(aligned, title = "Cross-Domain MSA") ##Load webpage of MSA

aa_prokaryotic <- readAAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\aa_prokaryotic.txt")
aligned_prokaryotes <- AlignSeqs(aa_prokaryotic)
BrowseSeqs(aligned_prokaryotes, title = "MSA of Prokaryotes")

aa_eukaryotic <- readAAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\aa_eukaryotic.txt")
aligned_eukaryotes <- AlignSeqs(aa_eukaryotic)
BrowseSeqs(aligned_eukaryotes, title = "MSA of Eukaryotes")
```

Cross-Domain Multiple Sequence Alignment



Distance Matrix

	Methanopyrus kandleri WP_011018374.1	Hyperthermus butylicus WP_011822242.1	Pyrobaculum aerophilum WP_011009153.1	Pyrococcus horikoshii WP_048053494.1
Methanopyrus kandleri WP_011018374.1	0.000000	1.864785	1.739703	1.4552872
Hyperthermus butylicus WP_011822242.1	1.864785	0.000000	2.190256	1.7850705
Pyrobaculum aerophilum WP_011009153.1	1.739703	2.190256	0.000000	1.7112206
Pyrococcus horikoshii WP_048053494.1	1.455287	1.785070	1.711221	0.0000000
Thermoproteus uzoniensis WP_013680739.1	1.408767	1.827350	1.425825	1.3298531
Thermococcus kodakarensis WP_048053707.1	1.609438	1.609438	1.616082	0.3352608
Staphylothermus marinus WP_011839562.1	2.369525	1.091923	1.997203	1.9143618
Vulcanisaeta moutnovskia WP_052298529.1	1.544899	2.018406	1.860752	1.2407868
Desulfurococcus amylolyticus WP_014767755.1	2.012602	2.047693	1.649789	1.3911843
Methanothermus fervidus DSM 2088 ADP78051.1	1.575536	1.629916	1.825892	1.0284080
Geoglobus ahangari WP_048096401.1	1.869318	1.906689	1.634131	1.3520612
Thermococcus henrietii WP_099211889.1	1.521469	1.644529	1.643339	0.3437715
Thermococcus guaymasensis WP_062370336.1	1.501224	1.609438	1.649984	0.3268220
Methanocaldococcus villosus WP_004590579.1	1.678431	1.881786	1.819158	1.4044217
Palaeococcus pacificus WP_048164940.1	1.598265	1.745850	1.704748	0.3019244
Spatholobus suberectus TKY74878.1	1.615869	2.008214	1.926237	1.7589696
Quillaja saponaria KAJ7966392.1	1.655048	2.008214	1.839226	1.7917595
Acaulospora colombiana (5242) CAG8617845.1	2.067654	1.906170	2.331335	2.0683509
Acaulospora morrowiae CAG8442076.1	1.409825	1.717651	1.734601	1.3862944
Sesamum alatum KAK4436377.1	1.680534	2.261763	1.609438	1.7479568
Perilla frutescens var. frutescens KAH6816902.1	1.762772	2.197225	1.571698	1.7197860
Linnemannia elongata KAK5808606.1	1.673976	2.079442	1.845827	1.6208017
Mortierella sp. 14UC KAF9118983.1	1.641187	2.079442	1.805005	1.6796422
Lotus japonicus XP_057421610.1	1.609438	2.295896	2.059507	1.6320377
Ziziphus jujuba XP_015873951.2	1.680207	2.113575	1.959424	1.5497996
Gigaspora margarita CAG8819886.1	1.529885	1.966113	1.854938	1.5012243
Acaulospora colombiana (1035) CAG8438707.1	1.500898	1.966113	1.778336	1.4759065
Dissophora ornata KAI8598821.1	1.729239	2.028148	1.845827	1.6687545

Continues for the entire matrix, as this is only a portion. The full output can be included in the Rmarkdown included with my submission.

Poisson Correction

Poisson Correction (PC) distance

The Poisson correction distance assumes equality of substitution rates among sites and equal amino acid frequencies while correcting for multiple substitutions at the same site.

MEGA provides facilities to compute the following quantities:

Quantity	Description
d: distance	Number of amino acid substitutions per site.
L: No of valid common sites	Number of sites compared.

Formulas used are:

Quantity	Formula	Variance
d	$-\ln(1-p)$	$p/[(1-p)L]$

Poisson Correction (PC) distance

Code for Distance Matrix & PCA

```
## Distance Matrix and Principal Component Analysis


```{r}
dists <- DistanceMatrix(aligned, type = "matrix", correction = "Poisson") #DECIPHER
View(dists) ##Create distance matrix with Poisson correction method
class(dists)

##Perform Principal Component Analysis - base R
pca_result <- prcomp(dists, scale. = TRUE)

# Create a data frame from PCA results
pca_df <- data.frame(
  PC1 = pca_result$x[,1],
  PC2 = pca_result$x[,2],
  Label = rownames(dists)
)
class(pca_df)

# Plot using ggplot2
ggplot(pca_df, aes(x = PC1, y = PC2, label = Label)) +
  geom_point(color = "blue", size = 2) +
  geom_text_repel(size = 3, box.padding = 0.4, point.padding = 0.3, max.overlaps = 50) +
  theme_minimal() +
  labs(title = "PCA of Sequence Distances",
       x = "PC1", y = "PC2")

summary(pca_result)
head(pca_result$x)
```

Create a data frame with the first and second principal component scores for each sequence, labelled with the row names from the distance matrix

Create a dot plot with PC1 and PC2 as x and y axes, with sequence IDs as labels. Format by avoiding overlapping text, label the aces and the title, and use blue dots for each point.
```

Code for Phylogenetic Tree & Bootstrapping

```
##Phylogenetic Tree

```{r}
set.seed(42)

tree_upgma <- upgma(dists)
plot(tree_upgma, main = "UPGMA Phylogenetic Tree")

bootstrap_trees <- boot.phylo(tree_upgma, dists, FUN = function(xx) upgma(dist.aa(xx)), B = 100, trees = TRUE)
print(bootstrap_trees) ##Run bootstrapping of the tree w given distance matrix using 100 iterations and UPGMA formula

consensus_tree <- consensus(bootstrap_trees$trees, p = 0.39, check.labels = TRUE, rooted = TRUE)
consensus_tree <- addConfidences(consensus_tree, bootstrap_trees$trees)

# Add to plot
plot(consensus_tree, main="Bootstrap Consensus Tree- UPGMA", type="phylogram", cex = 0.8, cex.lab = 1.5, show.tip.label = TRUE)
nodelabels(consensus_tree$node.label, frame = "none", cex = 0.8, cex.lab = 1.5, col = "red")

tree_nj <- nj(dists)
plot(tree_nj, main = "Neighbor Joining Phylogenetic Tree")

nj_bootstrap_trees <- boot.phylo(tree_nj, dists, FUN = function(xx) nj(dist.aa(xx)), B = 100, trees = TRUE)
print(nj_bootstrap_trees) ##Run bootstrapping of the tree w given distance matrix using 100 iterations and UPGMA formula

nj_consensus_tree <- consensus(nj_bootstrap_trees$trees, p = 0.45, check.labels = TRUE, rooted = FALSE)
nj_consensus_tree <- addConfidences(nj_consensus_tree, nj_bootstrap_trees$trees)

# Add to plot
plot(nj_consensus_tree, main="Bootstrap Consensus Tree- Neighbor-Joining", type="phylogram", cex = 0.8, cex.lab = 1.5, show.tip.label = TRUE)
nodelabels(nj_consensus_tree$node.label, frame = "none", cex = 0.8, cex.lab = 1.5, col = "red")

```
```

Set a seed for reproducibility since bootstrapping requires a random process.

Construct and plot a phylogenetic tree from a distance matrix using the UPGMA method

Perform bootstrapping on the initial phylogenetic tree from the distance matrix. FUN gives a custom function to convert the bootstrapped sequences into a distance matrix, and build a tree. 100 iterations are performed.

Build a consensus tree from the 100 bootstrap trees. A branch must be in at least 39% of trees to be included. Return a rooted tree, with the bootstrap values (addConfidences) as labels.

Repeat process with the neighbor-joining method and a threshold of 45%.

Code for GC Content Analysis

```
##GC Content Analysis

```{r}

dna_sequences <- readDNAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\projectfastadnaseq.txt")
  print(dna_sequences)
gc_content <- letterFrequency(dna_sequences, letters = c("G", "C"), as.prob=TRUE)
gc_total <- rowSums(gc_content)
print(gc_total)
summary(gc_total)
```

```
 [1] 0.5988372 0.5268065 0.4935401 0.4431138 0.5530303 0.5843621 0.3634752 0.4125230 0.4559585 0.3081761 0.5480226 0.5734990 0.5720165 0.3109620 0.4383838
[16] 0.5371550 0.4249714 0.3870023 0.3577982 0.4307562 0.6165803 0.5424028 0.5696203 0.3844156 0.4299754 0.3192661 0.4141049 0.4711965
```

Load sequences into DNAStringSet object. Call letterFrequency, focusing on letters G and C, on the DNA sequences, and store as proportion.

The columns in the GC Content Matrix are G% and C%, so we add the two using rowSums to get the GC% for each sequence. Store as its own object, and print. Include the statistical summary.

Each of the sequences' GC content as a proportion of their total nucleotides is represented.
Median = 0.4495
Mean = 0.4667
Range = [0.3082, 0.6166]
Only one sequence is particularly GC-rich.
```

Code for GC Content Analysis

```
dna_prokaryotic <- readDNAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\dna_prokaryotic.txt")
dna_eukaryotic <- readDNAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\dna_eukaryotic.txt")

gc_prok <- rowSums(letterFrequency(dna_prokaryotic, c("G", "C"), as.prob=TRUE)) * 100
gc_euk <- rowSums(letterFrequency(dna_eukaryotic, c("G", "C"), as.prob=TRUE)) * 100

gc_data <- data.frame(
  GC_Content = c(gc_prok, gc_euk),
  Group = c(rep("Prokaryotes", length(gc_prok)), rep("Eukaryotes", length(gc_euk)))
)

ggplot(gc_data, aes(x=Group, y=GC_Content, fill=Group)) +
  geom_boxplot() +
  scale_y_continuous(limits = c(0, 100))+  # Y-axis from 0 to 1

  theme_minimal() +
  labs(title="GC Content Comparison", y="GC Content (%)")

t.test(gc_prok, gc_euk)
```

Load sequences into DNAStringSet object – one for prokaryotic sequences, one for eukaryotic sequences. Call letterFrequency, focusing on letters G and C, on the DNA sequences, and scale by 100 for percentage.

Create a data frame object with the GC% calculated above. Combine the numeric vectors for each group (prokaryotes and eukaryotes). Create a group labelled "Prokaryotes" repeated as many times as there are prokaryotic sequences (entries in GC% object), and do the same for "Eukaryotes".

Plot the information in the data frame as a boxplot, where the x-axis is the group defined in the data frame, and the y-axis is the GC content defined in the data frame. Scale to be on a y-axis of of 0 to 100 (%) to create a fair visualization. Then perform a t-test.

Code for Codon Usage Bias

```
## Codon Usage Bias

```{r}


cds_prokaryotic <- readDNAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\cds_prokaryotic.txt")
seq_prokaryotic <- as.character(cds_prokaryotic)

cds_eukaryotic <-  readDNAStringSet("C:\\Users\\sophi\\OneDrive\\Documents\\ClassAssignments\\BIOL4815\\cds_eukaryotic.txt")
seq_eukaryotic <- as.character(cds_eukaryotic)

# Function to clean sequences
clean_seqs <- function(seqs) {
  seqs <- toupper(seqs)
  seqs <- seqs[nchar(seqs) %% 3 == 0]  # Keep sequences with length divisible by 3
  return(seqs)
}

# Clean sequences
seqs_prok_clean <- clean_seqs(seq_prokaryotic)
seqs_euk_clean  <- clean_seqs(seq_eukaryotic)

codon_prok <- count_codons(seqs_prok_clean)
codon_euk <- count_codons(seqs_euk_clean)

rscu_prok <- est_rscu(codon_prok)
rscu_euk  <- est_rscu(codon_euk)

# Merge RSCU data frames
rscu_combined <- merge(rscu_prok, rscu_euk, by = "codon", suffixes = c("_prok", "_euk"))

ggplot(rscu_combined, aes(x = rscu_prok, y = rscu_euk, label = codon)) +
  geom_point(color = "darkblue") +
  geom_text(size = 2, vjust = -0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Relative Synonymous Codon Usage, Thermophilic Prokaryotes and Mesophilic Eukaryotes",
       x = "Prokaryotes RSCU",
       y = "Eukaryotes RSCU") +
  theme_minimal()


```
```

Read CDS FASTA files for each eukaryotic and prokaryotic sequences, and convert to string of characters.

"Clean" sequences (help correct errors and put in right format for cubar functions). Convert to all uppercase and filter out any sequences whose length isn't a multiple of 3 (number for codon). Apply the function to the DNAStringSet objects.

Call count_codons on the cleaned sequences to scan each sequence and create a data frame of how many times each codon appears. Using this information, call est_rscu to compute RSCU and return a data frame. Then merge the two (inner join), and add labels (_prok/_euk) to distinguish the two.

Plot the combined RSCU data frame as a dot plot, with labels for each codon. Add a reference line where RSCU = 1.